

Coverages and the DAP2 Data Model

James Gallagher

22 March 2012

Outline

- DAP2 data model
- Servers and their relation to the data model
- Coverages and DAP2
- Support for Quadrilateral Grids
- Other kinds of grids
- Example: UGrid work with NOAA

The Data Model versus Servers

- Data Model
 - Variables
 - Operations
 - Datasets
- Servers
 - Software that implements the operations and which provides other features
 - Expressions combine variables, referenced by name, and operators to *constrain* access.

Variables and their Operations provide a collection of Abstract Datatypes. URLs provide unique references to Datasets and Servers used to access them.

The Data Model and Application Domains

- DAP2 is domain neutral
- DAP2 is intended to be a lower level in a multi-tier protocol stack
- Examples of standards often used with DAP2:
 - COARDS
 - CF
 - HDF-EOS; HDF-EOS2
- The separation is intentional!

Data Model

- Variables:
 - Name, Type, Shape: syntactic metadata
 - Attributes (name-type-value triples): semantic metadata
 - Value(s)
- Operations:
 - The operations that can be performed on the variables (subset, sample and function invocation)
- Datasets:
 - Instantiations of Variables

Datatypes in DAP2

- Types found in a ‘typical’ programming language
 - Structures: define new lexical scopes; contain one or more other variables; recursive
 - Arrays: type-homogeneous; indexed; n-dimensional; includes arrays of structures
 - Scalars: the fundamental building blocks; Byte, ..., String, URL.
- Sequence: Relational type; may be nested; defines a lexical scope; like structure, can contain one or more other variables; recursive (so nesting of relations is possible);
- Grid: Similar to a ‘quadrilateral coverage;’ defines a lexical scope; is *not* recursive.

Aside: Lexical Scope

```
Structure {  
    Int32 x;  
    Structure {  
        Int32 x, y;  
    } point;  
} x[1024];
```

- Lexical scoping prevents name ‘collisions’
- Provides hierarchy
- Provides grouping

Aside: Lexical Scope

```
Structure {  
    Int32 x;  
    Structure {  
        Int32 x, y;  
    } point;  
} x[1024];
```

- The above example defines 3 lexical scopes:
 - The outermost, which *contains* the Structure *x*
 - The things contained by the Structure *x*
 - The things contained by the Structure *point*
- Some Fully Qualified Names:
 - Assume that the dot operator has its typical meaning...
 - *x*, *x.x*, *x.point.x*

Aside: Lexical Scope

```
Structure {  
    Int32 x;  
    Structure {  
        Int32 x, y;  
    } point;  
} x[1024];
```

- The above example defines 3 lexical scopes:
 - The outermost, which *contains* the Structure *x*
 - The things contained by the Structure *x*
 - The things contained by the Structure *point*
- Some Fully Qualified Names:
 - Assume that the dot operator has its typical meaning...
 - *x*, *x.x*, *x.point.x*

Aside: Lexical Scope

```
Structure {  
    Int32 x;  
    Structure {  
        Int32 x, y;  
    } point;  
} x[1024];
```

- The above example defines 3 lexical scopes:
 - The outermost, which *contains* the Structure *x*
 - The things contained by the Structure *x*
 - The things contained by the Structure *point*
- Some Fully Qualified Names:
 - Assume that the dot operator has its typical meaning...
 - *x*, *x.x*, *x.point.x*

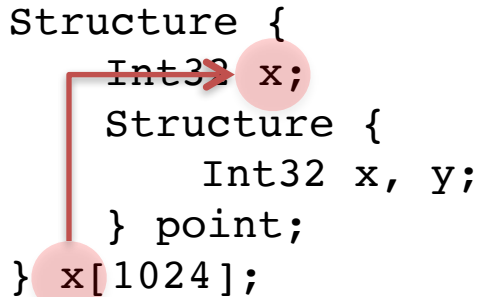
Aside: Lexical Scope

```
Structure {  
    Int32 x;  
    Structure {  
        Int32 x, y;  
    } point;  
} x[1024];
```

- The above example defines 3 lexical scopes:
 - The outermost, which *contains* the Structure *x*
 - The things contained by the Structure *x*
 - The things contained by the Structure *point*
- Some Fully Qualified Names:
 - Assume that the dot operator has its typical meaning...
 - *x*, *x.x*, *x.point.x*

Aside: Lexical Scope

```
Structure {  
  Int32 x;  
  Structure {  
    Int32 x, y;  
  } point;  
} x[1024];
```



- The above example defines 3 lexical scopes:
 - The outermost, which *contains* the Structure *x*
 - The things contained by the Structure *x*
 - The things contained by the Structure *point*
- Some Fully Qualified Names:
 - Assume that the dot operator has its typical meaning...
 - *x*, *x.x*, *x.point.x*

Aside: Lexical Scope

```
Structure {  
    Int32 x;  
    Structure {  
        Int32 x, y;  
        point;  
    } x[1024];  
}
```

The diagram illustrates the lexical scope structure of the provided code. Red circles highlight the following elements: the opening curly brace of the outermost structure, the variable `x` in the inner structure's parameter list, the variable `point`, the closing curly brace of the inner structure, and the array access `x[1024]`. Red arrows indicate the scope containment: one arrow points from the outermost brace to the inner structure's brace, and another points from the inner structure's brace to the `point` variable.

- The above example defines 3 lexical scopes:
 - The outermost, which *contains* the Structure `x`
 - The things contained by the Structure `x`
 - The things contained by the Structure `point`
- Some Fully Qualified Names:
 - Assume that the dot operator has its typical meaning...
 - `x`, `x.x`, `x.point.x`

Operations

- Operations provide a way to extract portions of a dataset to optimize data transfers – clients know what they want and can ask for that and no more
- Operations are made up of variable names and operators and/or function calls
- Operations can subset and sample arrays; extract relations from Sequences using relational operators; and call functions to compute new values using data from one or more variables.

Constraint Expressions

- Used to select individual variables
- Several variables at once
- Slice and sample arrays
- Select elements (rows) from Sequences
- Call functions
- Compose functions

Constraint Expression Examples

- SST
- SST.TIME
- SST.TIME,SST.COADSX
- SST.TIME[3]
- SST.TIME[2:2:8]
- geogrid(SST,45,-82,40,-78)
- linear_scale(geogrid(SST,45,-82,40,-78),10,0)

```
Dataset {  
  Grid {  
    Array:  
      Float32 SST[12][90][180];  
    Maps:  
      Float64 TIME[TIME = 12];  
      Float64 COADSY[COADSY = 90];  
      Float64 COADSX[COADSX = 180];  
    } SST;  
  } coads_climatology;
```

*The real dataset for these examples is at test.opendap.org/dap/data/nc/coads_climatology.nc.

e.g.: [test.opendap.org/opendap/data/nc/coads_climatology.nc.asc?linear_scale\(geogrid\(SST,45,-82,40,-78\),10,0\)](http://test.opendap.org/opendap/data/nc/coads_climatology.nc.asc?linear_scale(geogrid(SST,45,-82,40,-78),10,0))

Constraint Expression Examples:

Sequences support relational ops

- &cruise=1
- &cruise={1,3,5}
- &cruise<6
- &lat,lon
- lat,lon&cruise=1

```
Dataset {  
    Sequence {  
        Sequence {  
            Float32 lat, lon;  
            Sequence {  
                Int32 depth;  
                Float32 temp;  
            } cast;  
        } leg;  
    } cruise;  
} XBT_Data;
```

Datasets

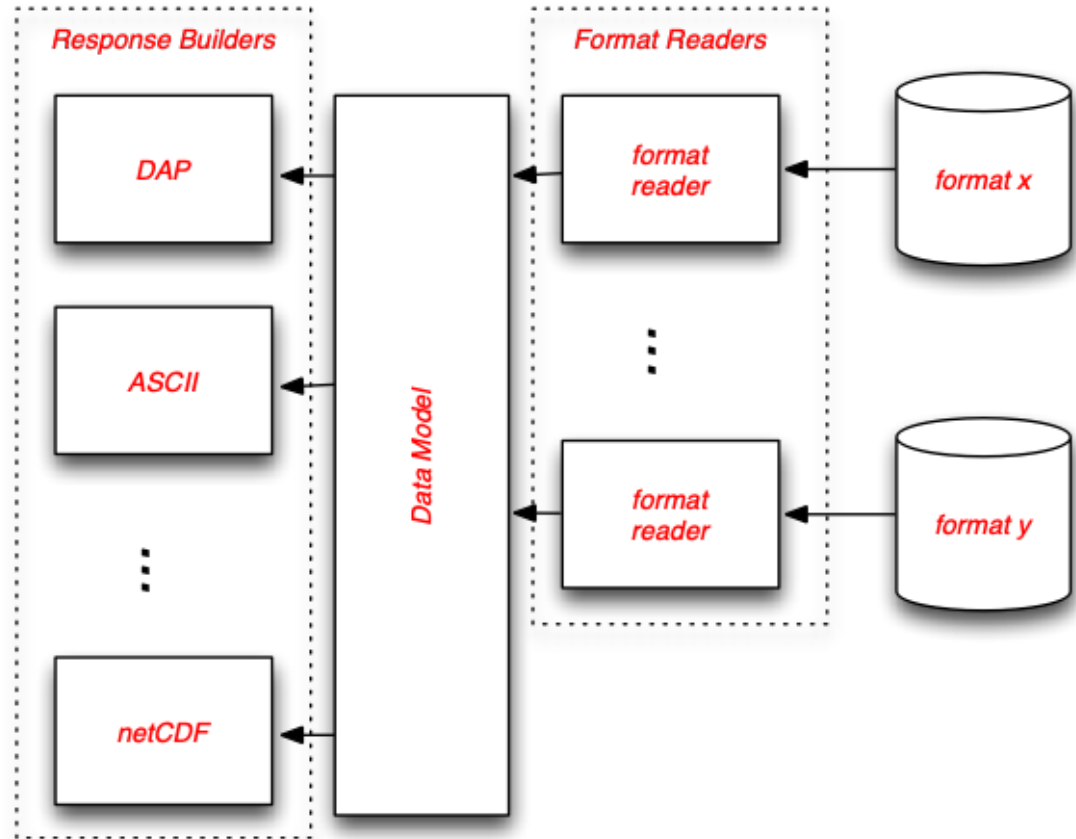
- Collections of variables
- Each Dataset is held in a single Structure instance that holds one or more variables
- Attributes defined at this top-most level and not explicitly bound to a variable are “global attributes” and convey information about the dataset as whole
- Each variable in the dataset has zero or more attributes.
- Attributes are lexically scoped.

OPeNDAP Servers often do more

- DAP2 specifies two metadata responses
- ...and one data response (BLOB; XDR encoded)
- Servers also typically return/provide:
 - THREDDS catalogs plus HTML pages
 - XML, RDF encodings for metadata
 - ASCII, netCDF encodings for data

data model = software bus

- *Software Bus*: A programming interface that allows software modules to transfer data to each other.
- Format readers decode data and produce a representation using the data model
- Response builders use the data model representation
- This mirrors the larger client server design where clients can read data stored in many different formats from one or more servers.



Summary of the Data Model

- Variables: Datatypes provide 'syntactic metadata; attributes provide semantic metadata
- Datasets: Concrete collections of variables
- Operations: Provide a way to use operators to extract parts of variables; based on projection, selection and function invocation
- Servers provide a web interface using the data model

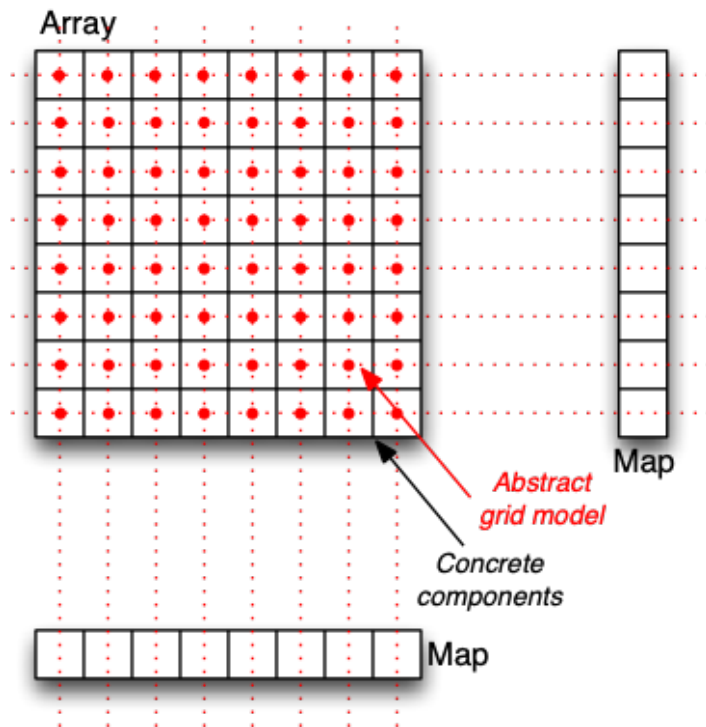
Coverages and DAP2

- DAP2 'Grid' provides the structure needed for a quadrilateral grid
- DAP2 is 'domain neutral'
- Information about geospatial location should be provided by attributes

DAP2's Grid Data type

- ...is a Quadrilateral Grid
- “DAP2 Grid” “Maps”
 - Define the coverage extent
- “DAP2 Grid” “Array”
 - Define the schema mapping
- Grid lines do not have to be equally spaced
- N-dimensional
- Limitations
 - Grid lines only; curves are not supported
 - All Grid lines intersect at right angles

DAP2 Grids: Abstract and Concrete Representations



- Array: Holds the ‘feature attribute value records.’ These are accessed using integer indices $i_0, i_1, \dots, i_n$
- Map: Holds the grid points where the value of the i_0^{th} grid point is i^{th} value of the m_0^{th} vector.
- Because the DAP2 Grid datatype uses vectors for its Maps, the Grid datatype is an ‘orthogonal grid.’

Continuous versus Discrete...

- DAP2 provides a mechanism
- It does not specify policy
- Thus, 'Grid' can represent a variety of data abstract grid coverages.
- The exact mapping – e.g., how to (or if) interpolate values 'between the grid points' – must be provided in attributes or by convention.

DAP2 Grid compared to CV_Grid

- *Figure 16 – CV_Grid.* From “The OpenGIS Abstract Specification Topic 6: Schema for coverage geometry and functions,” OGC 07-001, p. 41

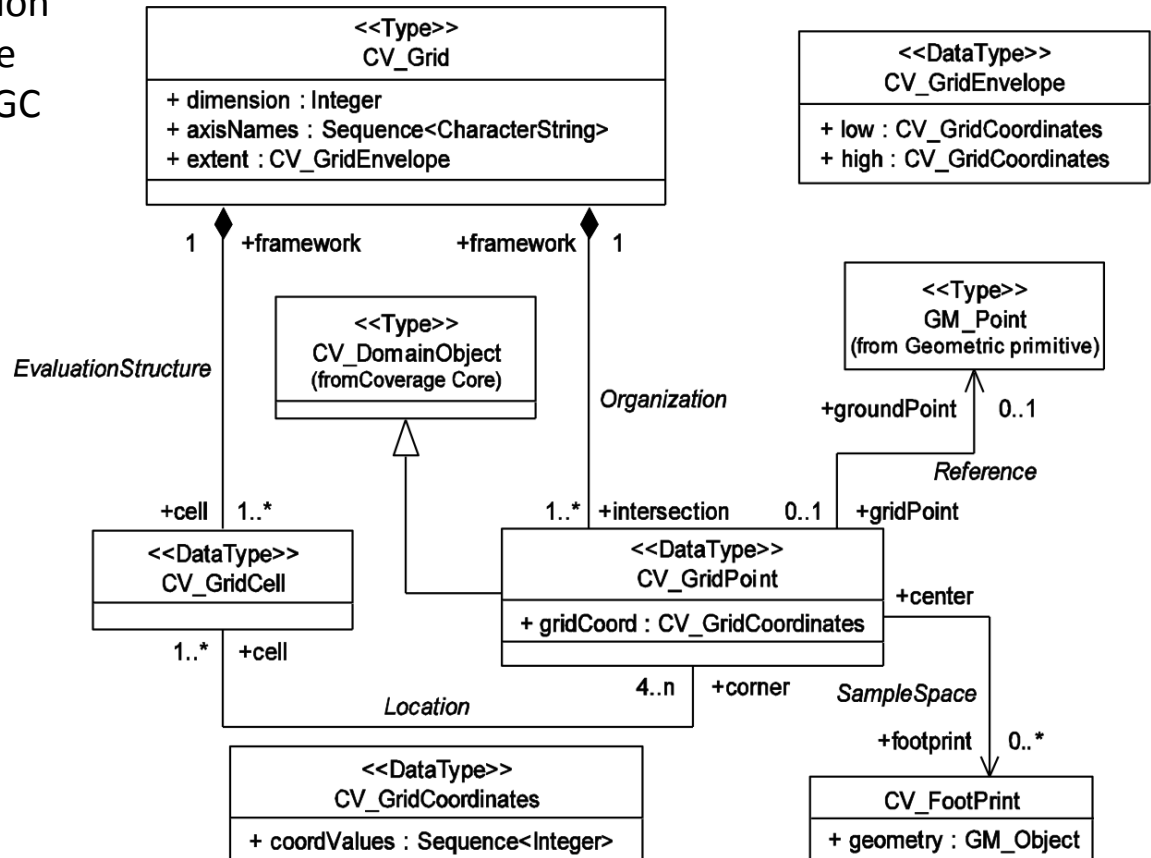


Figure 16 — CV_Grid

DAP2 Grid compared to CV_Grid

- *Dimension* and *axisNames* are present in the Grid variable's definition.
- *Extent* can be read from the Map vectors or from the Grid variable's attributes.
- The *GridEnvelope* can also be read from the Maps and Grid attributes if they are present.

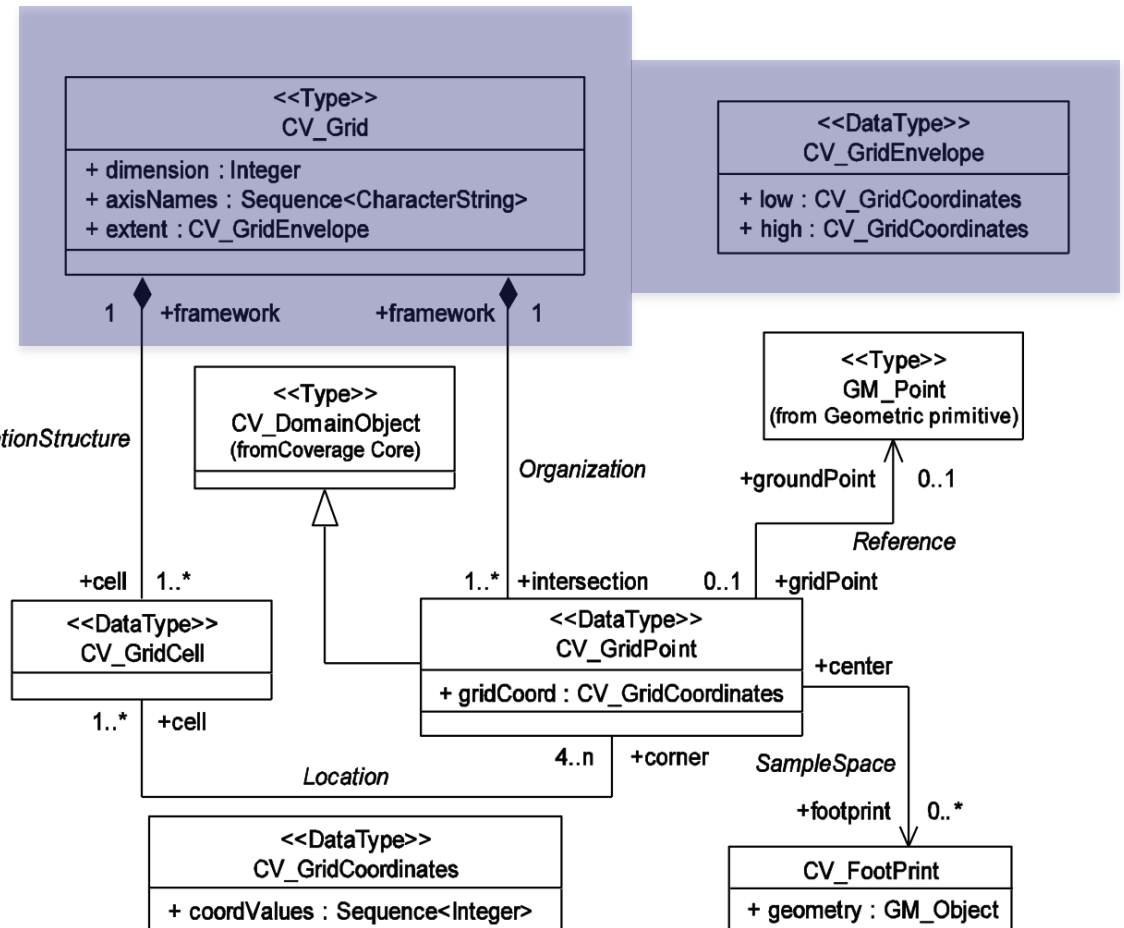


Figure 16 — CV_Grid

DAP2 Grid compared to CV_Grid

- The DAP2 Grid variable's Maps provide this information.
- The *EvaluationStructure* is implicit, however.
- The extent of the Grid can be read from the Maps

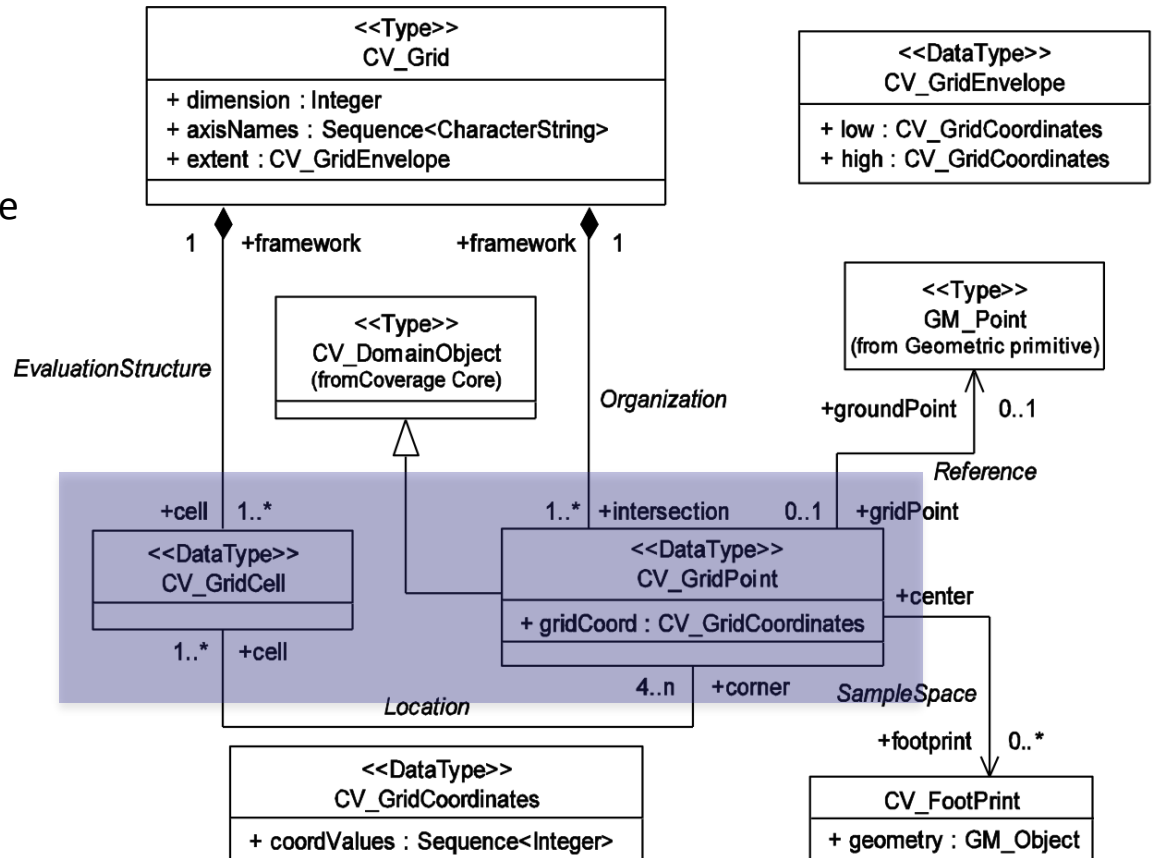


Figure 16 — CV_Grid

DAP2 Grid compared to CV_Grid

- *GridCoordinates* are held in the values stored in the Map's

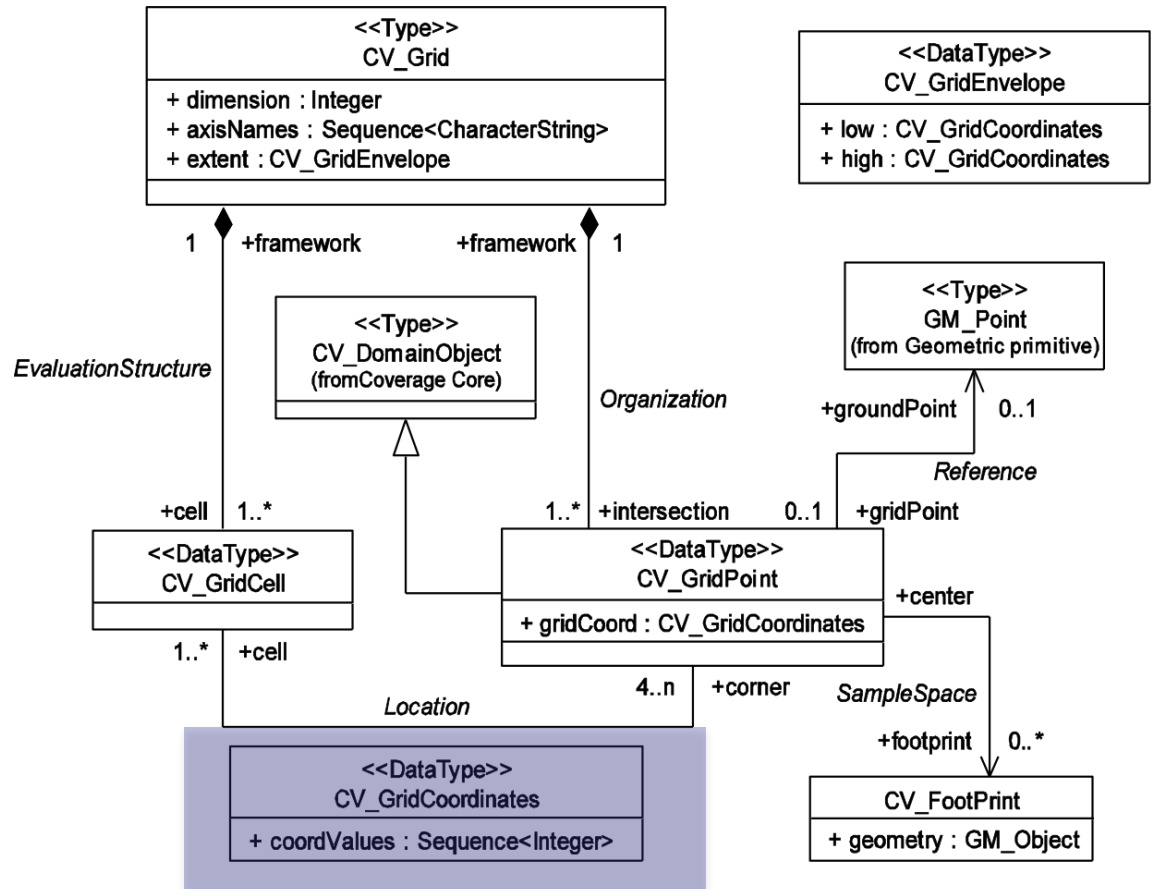


Figure 16 — CV_Grid

DAP2 Grid compared to CV_Grid

- Information about the footprint of the grid cell (as defined in sec 8.2.2 of OGC 07-011) may be provided in attributes of the Grid variable

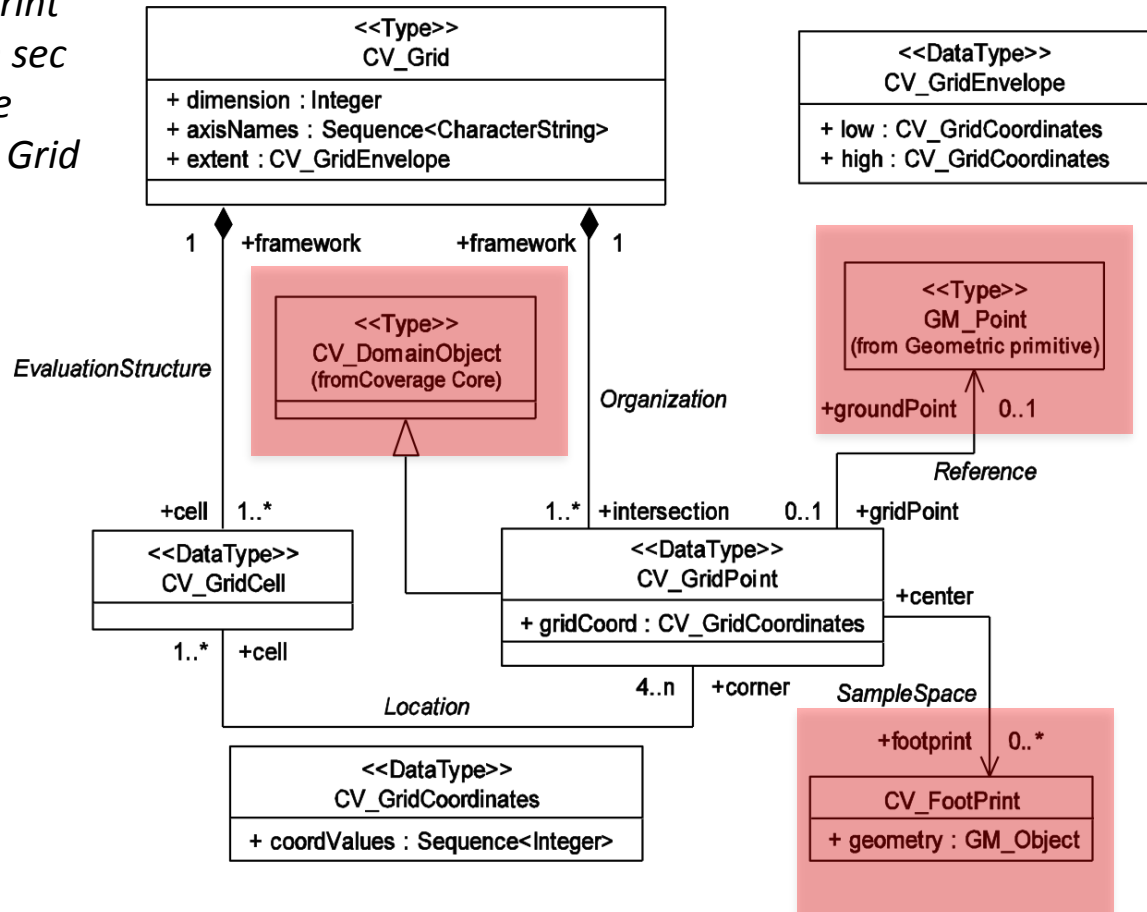


Figure 16 — CV_Grid

Representing Other Kinds of Coverages

- Assumptions:
 - Constructor types can build the necessary data structures
 - DAP2 Global and Variable attributes can encode interpretation information

Other Kinds of Coverages, cont.

- Hexagonal Grid
 - Can be implemented using DAP2's Grid variable and extra attribute information.
- Thiessen polygon
- Triangular irregular networks (TINs)
 - Both can be encoded using the conventions being developed by ASA & Deltares (http://bit.ly/ugrid_cf)

Summary

- DAP2 data model is domain neutral
- Provides structural support for quadrilateral grids
- Can be used to encode a wide variety of coverages

Domain neutral design can leverage other application-specific standards
Domain neutral design fosters cross-domain interoperability